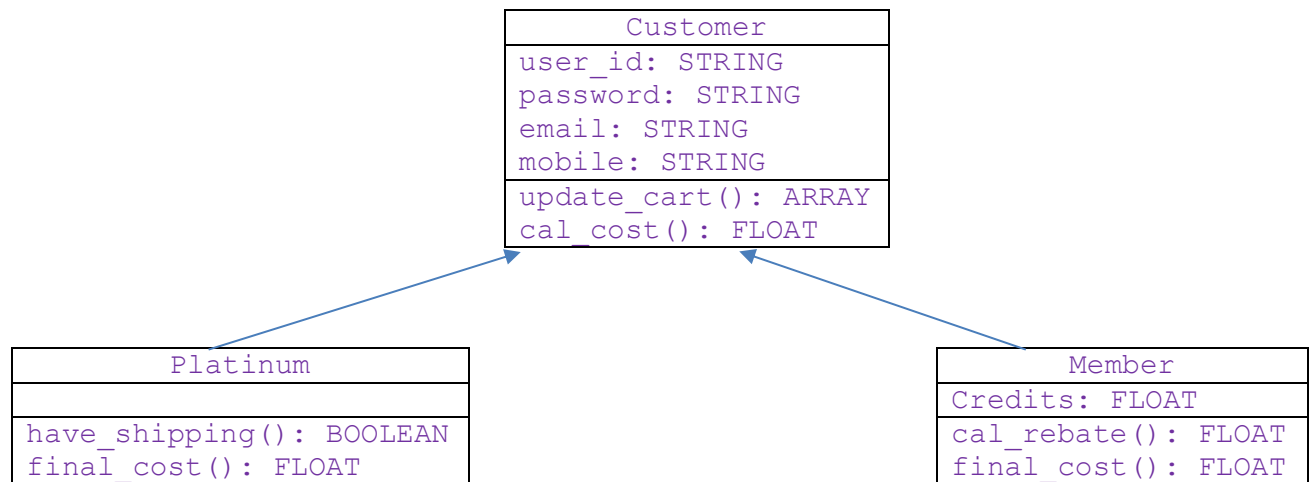


1

(a) Draw a UML class diagram to show the following for the situation described above:

- the superclass
- any subclasses
- inheritance
- properties
- appropriate methods.

[6]



(b) State the purpose of inheritance in OOP and explain how it is achieved, using the example above. [3]

- Reduce code redundancy
- Make use of existing code in other classes
- Attributes that are common in all subclasses such as `user_id`, `email`, `password` etc are defined in the superclass. They do not need to be redefined in the subclasses.

(c) Describe **one** situation that could threaten the privacy of customer data that the platform has collected and state **one** measure the e-commerce platform should take to protect the data that it has collected. [2]

- Access to the data by unauthorised persons.
- Authenticated access to data.

(d) As e-commerce becomes more and more popular, describe **one** social impact of this trend. [2]

- Social: loss of inclusivity for citizens who have difficulties using online services, e.g. elderly who may not be familiar with using such applications, etc.

2

- (a) State **one** usability principle and describe how the user interface of the registration and login module of the membership portal should be designed to demonstrate this principle. [2]

Example: One usability principle is error prevention. When designing the login module, the system should incorporate an option to reveal what is typed in the password field so that the user can confirm the password before submitting.

Any valid pair in context of the above scenario:

#1: Visibility of system status

- The design should always keep users informed about what is going on, through appropriate feedback within a reasonable amount of time.

#2: Match between system and the real world

- The design should speak the users' language. Use words, phrases, and concepts familiar to the user, rather than internal jargon. Follow real-world conventions, making information appear in a natural and logical order.

#3: User control and freedom

- Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave the unwanted action without having to go through an extended process.

#4: Consistency and standards

- Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform and industry conventions.

#5: Error prevention

- Good error messages are important, but the best designs carefully prevent problems from occurring in the first place. Either eliminate error-prone conditions, or check for them and present users with a confirmation option before they commit to the action.

#6: Recognition rather than recall.

- Minimize the user's memory load by making elements, actions, and options visible. The user should not have to remember information from one part of the interface to another. Information required to use the design (e.g. field labels or menu items) should be visible or easily retrievable when needed.

#7: Flexibility and efficiency of use

- Shortcuts — hidden from novice users — may speed up the interaction for the expert user such that the design can cater to both inexperienced and experienced users. Allow users to tailor frequent actions

#8: Aesthetic and minimalist design

- Interfaces should not contain information which is irrelevant or rarely needed. Every extra unit of information in an interface competes with the relevant units of information and diminishes their relative visibility.

#9: Help users recognize, diagnose, and recover from errors

- Error messages should be expressed in plain language (no error codes), precisely indicate the problem, and constructively suggest a solution

#10: Help and documentation

- It's best if the system doesn't need any additional explanation. However, it may be necessary to provide documentation to help users understand how to complete their tasks.

- (b) As your team works on the login module of the membership portal, you realised that your team has failed to incorporate appropriate data or input validation checks as well as input verification.

Explain **one** input validation check that you would like to your team to incorporate in the login module. [1]

Example: The login module should test for the presence of the username and password. If either of the fields is missing, the module should output an appropriate error message prompting the user to enter the username and/or password.

Any one of the following checks with proper contextual explanation.

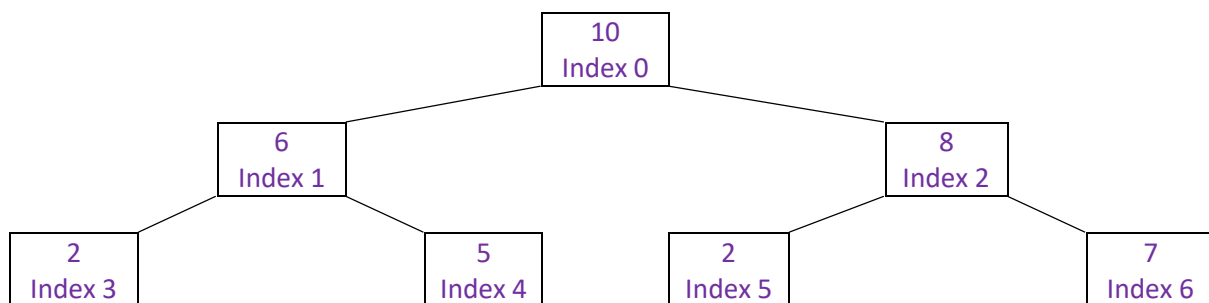
- Length check
- Range check
- Presence check
- Format check
- Check digit

3

- (a) Describe how inserting a new element is done in a sorted linked list, and state **one** possible advantage the above `insert()` method has over a sorted linked list. [3]

- In a sorted linked list, the element to be inserted is compared with all elements of lower (or higher) priorities before it is inserted in its position.
- In the `insert()` method above, the element to be inserted is only compared parent nodes that it is a subtree of. Hence, this method could be faster in certain cases.

- (b) Elements with priorities 10 and 7 are inserted into the priority queue in Figure 1. Sketch the final max heap. [3]



- (c) State **one** problem the suggested algorithm for `insert()` may cause. [1]

During the swapping the indices elements and their parent nodes, the elements with the same priorities may not be served in the same order that they are enqueued.

4

Describe your conversation with your team in deciding between a native application and a web-based application, by comparing **three** differences, in the context of the company's requirements. [3]

Any three differences in context of the above scenario:

Web Applications	Native Applications/Desktop Application
Deployment and maintenance (updates) for a web-based application require deployment on a single set of server machines.	Deployment and any maintenance/patch are done on individual client machines separately.
Web applications can be accessed from anywhere (most locations), so there is no location constraint.	As desktops are confined to a standalone machine, they can only be accessed from the machines they are deployed in.
Web applications are platform-independent, they can work in different types of platforms with the only requirement of a web browser.	Desktop applications need to be developed separately for different platform machines. (Windows, Linux, Unix, Mac etc)
Web applications are at higher security risks as they are inherently designed to increase accessibility.	Desktop applications, on the other hand, have better authorization and administrators have better control, hence more secure.
Web applications rely heavily on internet connectivity, for their operation.	Desktop applications don't require the internet for their operations. Some applications just require internet connectivity at the time of update.

5

- (a) Using the database tables CG, SCHOLARSHIP and STUDENT, present the data above in second normal form. Explain why the resulting schema is not in third normal form. [4]

2NF: Every non-primary key attribute is fully functionally dependent on the primary key.
Composite keys needed for SCHOLARSHIP and STUDENT.

CG(CGName, Tutor) [1]

SCHOLARSHIP(CGName, IndexNo, Scholarship) [1]

STUDENT(CGName, IndexNo, StudentName, CCA, CCATeacher, ContactNo) [1]

In 3NF, no non-primary key attribute is transitively dependent on the primary key. E.g. in STUDENT table, CCA and CCATeacher are transitively dependent on CGName and IndexNo. [1]

- (b) Hence, present your schema in (a) in the third normal form. [2]

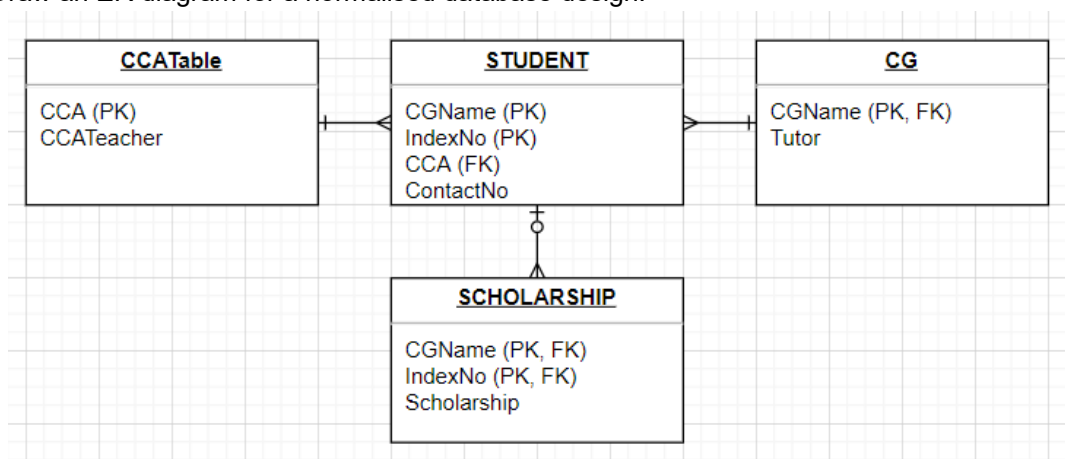
CCATable(CCA, CCATeacher) [1]

STUDENT(CGName, IndexNo, StudentName, CCA, ContactNo) [1]

SCHOLARSHIP(CGName, IndexNo, Scholarship)

CG(CGName, Tutor)

- (c) Draw an ER diagram for a normalised database design. [3]



- (d) By stating examples in the above context, explain the significance of the following terms:

- (i) primary key [2]

A primary key is a column or a set of columns in a table whose values uniquely identify a row in the table. [1]

With references to part (a) [1]

- (ii) foreign key [2]

A foreign key is a column or a set of columns in a table whose values correspond to the values of the primary key in another table. [1]

With references to part (a) i.e. CCA in STUDENT; CGName and IndexNo in SCHOLARSHIP [1]

- (e) State a scenario where a NoSQL database would be more appropriate for the above context. [1]

Scholarship can be optional. [Not acceptable because SCHOLARSHIP table has been defined with 1 or 0 relationship.]

E.g. The JC allows students to join more than one CCA.